Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kapsamı ve kaynakları (ekrandan, 00:10:00 ve 00:14:50)
  - Ders iki kısımdan oluşur: Kısım 1 Gang of Four (Dörtlü Çete) tasarım kalıpları, Kısım 2 Code Smells and Refactoring (birim sınamaları ile). Dönemin yaklaşık üçte ikisi tasarım kalıplarına, üçte biri kod kusurları ve refactoring'e ayrılır.
  - Birinci kısım işin doğrusunu, ikinci kısım kötü tasarımın nasıl fark edilip düzeltileceğini öğretir. Hoca iki kısmın ayrık olmadığını, birbirini tamamladığını vurguladı.
  - Kısım 1'in temel kaynağı: *Design Patterns: Elements of Reusable OO Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994. Zamanla literatüre yeni kalıplar eklenmiştir, ama GoF kalıpları en temel kümedir.
  - Kısım 2'nin kaynağı: *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999. Kitaptaki kusurların tamamı değil, en sık kullanılan ve birbiriyle ilişki ağı en yoğun olan bir alt küme işlenir.
  - Ek kaynaklar (slayttaki liste): *Design Patterns Explained* (Shalloway, Trott, 2004), *Head First Design Patterns* (Freeman vd., 2004), *Design Patterns* (Lasater, 2006), *Design Patterns Java Workbook* (Metsker, 2002), *Applied Java Patterns* (Stelting, Maassen, 2002).
  - GoF kitabı kalıbın tanımını ve içeriğini anlatır; öteki kitaplar kalıbın örnekler üzerinden kurulumunu ve gerçeklenmesini gösterir. İki türü de çalışmak yararlıdır.
- Kalıpların derste örneklenme biçimi
  - Bazı kalıplar için aynı örneğin üç çözümü verilir: akla ilk gelen ve sonra kullanışsız olduğu anlaşılan çözüm, kalıp kullanmadan nesneye yönelimi doğru kullanan çözüm, tasarım kalıbı uygulanan çözüm. Bu çözümler çeşitli açılardan karşılaştırılır.
  - Birkaç çok basit kalıp dışında her kalıp en az bir örnekle gösterilir.
- Ön koşullar (ekrandan, 00:14:50)
  - Herhangi bir güncel nesneye yönelik programlama diline hakimiyet. Derste kod örnekleri Java ile verilir. C# söz dizimi Java'ya çok benzer; dillerin ileri özellikleri kullanılmaz.
  - Kalıtım, soyut sınıflar ve arayüzler nesneye yönelimin temelleridir ve iyi bilinmelidir.
  - UML sınıf şemalarına hakimiyet. Önerilen kaynaklar: *UML Distilled* (Martin Fowler, 3. baskı, 2003), *UML 2.0 in a Nutshell* (Pilone, Pitman). Head First serisinin de UML kitabı vardır.
- Yazılım kalitesi (ekrandan, 00:40:00)
  - Amaç "iyi" tasarım ile "kaliteli" yazılıma ulaşmaktır.
  - Slayttaki tanım: problem alanı hangi düzeyde parçalara ayrılabiliyorsa o düzeyde parçalara ayrılabilen tasarım "iyi" sayılır. Hoca bu tanımı öznel buldu.
  - Dış kalite özellikleri kullanıcıya yöneliktir: doğruluk, etkinlik (kaynakları en az düzeyde kullanma), kullanım kolaylığı.
  - İç kalite özellikleri programcıya yöneliktir: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı. Ders iç kalite özellikleriyle ilgilenir.
- Neden iyi tasarım? (ekrandan, 00:41:30, 00:45:20 ve 00:48:20)
  - Yazılım projeleri önemli oranda başarısız olur (1950-60'lardan beri bilinen yazılım krizi). Etken çoktur, ama "tasarımın iyiliği" denetlenebilen bir etkendir.
  - Bir hatayı düzeltmenin bedeli yazılım hayat döngüsünde ilerledikçe üstel olarak artar. Slayta göre geliştirme çabasının çoğu bakım aşamasında harcanır.

  - Tam başarı: bütçe, süre ve işlev hedeflerinin üçüne de ulaşmak. “Zorlanılmış” proje: bu üçünden en az birinde başarısızlık.
  - Dr. Dobb’s Journal 2013 IT Project Success Survey: geleneksel (şelale) ve ad-hoc süreçlerde tam başarı yaklaşık %50, çevik, yinelemeli ve Lean süreçlerde yaklaşık %65-70.
  - Türkiye’de çevik projeler üzerine bir çalışma (E. Çetin, P. O. Durdu, UYMS 2014), 35 katılımcı. Sonuçlar da benzer düzeydedir.
- Düşük bağlaşım (low coupling) (ekrandan, 01:09:20, 01:20:00, 01:30:00, 01:32:15)
  - Bağlaşım, bir parçanın (sınıf, alt sistem, paket) diğer parçalara bağımlılık oranıdır. Derste sınıf düzeyinde ele alındı.
  - Bağımlılık nedenleri: diğer sınıfın hizmetlerinden yararlanmak, iç yapısından ya da çalışma ilkelerinden haberdar olmak, kalıtım ilişkisi.
  - Bağımlı olunan sınıfların sayısı arttıkça bağlaşım artar. Önemli olan nesne sayısı değil sınıf sayısıdır.
  - Yararları: bir sınıftaki değişiklik geri kalan sınıfların daha azını etkiler, yeniden kullanılabilirlik ve esneklik artar.
  - Ölçütler: sınıf bağlaşımı, toplam bağlaşım, maksimum bağlaşım, dolaylı ilişkilere 0,5 ağırlık.
- Yüksek uyum (high cohesion) (ekrandan, 01:40:00)
  - Uyum, bir parçanın sorumluluklarının birbiriyle uyumlu olma oranıdır: sorumlulukların aynı ya da çok yakın ilgi alanlarında olması.
  - Yararları: sınıfın anlaşılma kolaylığı artar, yeniden kullanılabilirlik artar, bakım kolaylığı artar, sınıfın değişikliklerden etkilenme olasılığı düşer.
  - Veritabanı örneği: veritabanı işlerini ayrı bir VT sınıfına vermek bağlaşımı artırır, ama uyumu yükseltir ve katmanlı mimari kurar.
  - Uyum, statik kod analiziyle metot-alan erişim örüntüsünden ölçülebilir.
- İlgi alanlarının ayrılması (separation of concerns) (ekrandan, 02:00:00)
  - Yazılımın yalnız belirli bir amaç, kavram ya da hedefle ilgili kısımlarının tanımlanabilmesi, birleştirilebilmesi ve değiştirilebilmesi yeteneğidir.
  - “Sorunu parçalayarak fethet ve parçaları yeniden kullan.” Parçaların düşük bağlaşıma ve yüksek uyuma sahip olması istenir.

Hocanın Özellikle Vurguladığı Kısımlar

- Nesneye yönelimin temelleri ve UML olmadan bu derse girilmemeli
  - Ders ustalık dönemidir; çıraklık dönemi, yani bir nesneye yönelik dilde problem çözüp sınıf tasarımı yapmak, önceden geçirilmiş olmalıdır.
  - UML sınıf şemasının temel kuralları nesneye yönelimin temellerine dayanır. Örneğin üye alanlar private, üye metotlar public tanımlanır; bu ilkelere aykırı bir şema hatalıdır.
- Bağlaşımda ilişkinin niteliğine değil varlığına bakılır
  - A’da B türünden tek bir alan olması ile B dizisi olması (bire bir ya da bire çok) aynı bağlaşımdır: A’nın kodu B’nin kodu olmadan kullanılamaz.
  - Kalıtım da bir bağlaşımdır. Çalışan sınıfı Kişi’den türetilmişse Kişi’nin kodu olmadan kullanılamaz.
- Kalıtım en az esnek ilişki türüdür
  - Deneyimli tasarımcılar bir ilişkiyi kalıtım yerine sahiplik gibi bir ilişkiyle modelleyebiliyorsa bunu tercih eder. Sahiplikte nesneler çalışma anında başka nesnelerle ilişkilendirilebilir; kalıtımla türetilen sınıf ise her zaman üst sınıfın bir türüdür.
- Dolaylı ilişkiler gözden kaçırılmamalı
  - A→B→C zincirinde A, C’ye dolaylı olarak bağımlıdır; A’yı yeniden kullanmak için C’nin kodu da gerekir.
- Bağlaşım ile uyum genelde aynı yönde rehberlik eder, ama ters düşebilir
  - Veritabanı örneğinde bağlaşımı düşük olan tasarım değil, uyumu yüksek olan tasarım (ayrı VT sınıfı) doğru çözümdür.
- Ölçütler çeliştiğinde karar geçmiş verilerle verilmeli
  - Toplam bağlaşım ile maksimum bağlaşım farklı tasarımı gösterebilir. Firma kendi geçmiş projelerinde hangi ölçütün kaliteyle (örneğin bin satır başına hata sayısı) daha çok ilişkili olduğuna bakarak seçim yapabilir.
- Tasarım problem alanını yansıtmalı

  - Sınıflar problem alanındaki varlıklarsa aralarındaki ilişki gerçek dünyadaki ilişki neyse odur; bağlaşım tartışması iki tasarımın da aynı problemi çözdüğü varsayımıyla yapıldı.

Kısa Tekrar Notları

- Kısım 1: GoF tasarım kalıpları (Gamma vd., 1994). Kısım 2: kod kusurları ve refactoring (Fowler, 1999).
- İyi tasarım: problem alanı hangi düzeyde parçalara ayrılıyorsa tasarım da o düzeyde parçalara ayrılır.
- Dış kalite kullanıcıya yönelik (doğruluk, etkinlik, kolaylık); iç kalite programcıya yönelik (yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik/anlaşılabilirlik, sınanabilirlik, bakım kolaylığı).
- Okunabilirlik ile anlaşılabilirlik farklıdır: okunan her şey anlaşılmaz. Tasarım kalıpları ortak bir sözlük sağlayarak anlaşılabilirliğe katkı verir.
- Yeniden kullanılabilirliğin çaba kazancı, hocanın okuduğu araştırmalara göre en fazla %30 civarındadır; kodu yeniden kullanılabilir yapmak için harcanan ek çaba kazancı azaltır.
- Esneklik: kodun benzer problemlere en az değişiklikle aktarılabilmesi ve gereksinim değişikliklerinin kolay karşılanması.
- Taşınabilirlik: aynı yazılımın farklı platformlarda (Windows'tan macOS'a, SQL Server'dan Oracle'a) kullanılabilmesi.
- Sınanabilirlik: bir şeyi düzgün test edemiyorsanız büyük ihtimalle doğru tasarlamamışsınızdır.
- Hata düzeltme bedeli hayat döngüsünde üstel artar; bu yüzden iyi bir tasarımla başlamak gerekir.
- Bağlaşım = bir parçanın diğer parçalara bağımlılık oranı. Düşük olması istenir.
- Bağlaşım hesabı: doğrudan ilişki 1, dolaylı ilişki 0,5; toplam ve maksimum bağlaşım iki ayrı sistem ölçütüdür.
- Uyum = sorumlulukların birbiriyle uyumlu olma oranı. Yüksek olması istenir.
- Uyum ölçümü: gerçekleşen metot-alan kullanımı / (metot sayısı × alan sayısı).
- Veritabanı işlerini ayrı sınıfa vermek: daha yüksek bağlaşım ama daha yüksek uyum ve katmanlı mimari. Doğru çözüm budur.

Detaylı Açıklamalar

- İç kalite özellikleri ve örnekler
  - İlgi alanlarının ayrılması: problem alanındaki alt gereksinimler, alt sorumluluklar hangi düzeyde ayrılabiliyorsa tasarım da onlara birebir karşılık gelmelidir. Bu ayrım ne kadar net yansıtılırsa çözüm o kadar kaliteli olur.
  - Esneklik için hocanın örneği: finans ve sağlık yazılımları ilk bakışta ayrı alanlardır. Ama finans yazılımı hisse ya da döviz değerlerini anlık izler, belli eşiklerde otomatik al-sat emri verir (algoritmik ticaret). Sağlık yazılımı da hayati göstergeleri (kan tahlili değerleri, yarım saatte bir ölçülen ateş) zaman içinde izler; değerin bir bant aralığında kalması gerekir. Zaman içinde değişen değeri izleme, analiz ve grafikleme kısmı küçük değişikliklerle bir alandan ötekine aktarılabilir.
  - Okunabilirlik ve anlaşılabilirlik: yazılımlar ekiplerce geliştirilir. Kodu ekibin öteki üyeleri, sınama ekibi ve sonraki sürümleri geliştirecek kişiler en az çabayla anlayabilmelidir. Açık kaynak projelerde farklı altyapı ve kültürden gelen katkıcılar için bu daha da önemlidir. Yalnız anlaşılabilirliği iyileştirmeye yönelik refactoring bile değerli bir refactoring'dir.
- Proje başarısı verileri (ekrandan, 00:45:20 ve 00:48:20)
  - Dr. Dobb's Journal 2013 anketinde süreç türlerine göre başarılı, zorlanılmış ve başarısız proje oranları çubuk grafikle gösterildi (Lean, Iterative, Agile, Ad-Hoc, Geleneksel). Ad-hoc, yazılım evinin literatürdeki süreçlerden seçerek ya da özgün olarak kendisinin kurduğu süreçtir; hoca bunun "özensiz" anlamına gelmediğini belirtti.
  - Başarı tanımı esnetilip zorlanılmış projeler de sayılırsa oran %90 civarına çıkar.
  - Türkiye çalışmasında "kurumunuzda gerçekleştirilen çevik projelerin başarı oranları" soruldu. Pasta grafiğindeki dağılım (katılımcı sayısı; yüzde): <%10 için 2 (%6),

%10-%25 için 4 (%12), %26-%50 için 5 (%14), 13 (%37), %76-%100 için 11 (%31).
  - Hoca 100'den 31'i çıkarıp 69 buldu ve bunu uluslararası %65-70 düzeyine yakın yorumladı.
  - Ders sonrası not: Hoca bu sayıyı "en fazla %69 başarı" diye okudu. Grafikteki %69 büyük olasılıkla bir başarı oranı değil, çevik projelerinin başarı oranı %75 ve altında kalan katılımcıların payıdır ($6 + 12 + 14 + 37 = 69$). Başka bir deyişle katılımcıların %31'i projelerinin %76-%100'ünün başarılı olduğunu bildirmiştir.
- Bağlaşım örneği: zincir ve yıldız tasarımları (ekrandan, 01:30:00 ve 01:32:15)
  - Tasarım I: A→B ve B→C (zincir). Tasarım II: A→B ve A→C (yıldız). İki tasarımın aynı problemi çözdüğü varsayılır.
  - Tasarım I: bağ(A) = 1, bağ(B) = 1, bağ(C) = 0. Toplam bağlaşım tBağ = 2, maksimum bağlaşım maxBağ = 1.
  - Tasarım II: bağ(A) = 2, bağ(B) = 0, bağ(C) = 0. tBağ = 2, maxBağ = 2.
  - Toplam bağlaşım fark göstermez. Maksimum bağlaşım (en riskli sınıfa bakan, savaş planlamasında en az kayıplı hareketi seçmeye benzeyen ölçüt) Tasarım I'i gösterir.
  - Dolaylı ilişki eklenince (A, B üzerinden C'ye bağımlı; 0,5 ağırlık, şemada kesikli çizgi): Tasarım I'de bağ(A) = 1,5, tBağ = 2,5, maxBağ = 1,5. Tasarım II değişmez (tBağ = 2, maxBağ = 2).
  - Sonuç çelişkilidir: toplam bağlaşıma göre II, maksimum bağlaşıma göre I daha iyidir. Karar, geçmiş projelerin bağlaşım ölçümleri ile kalite verisi karşılaştırılarak verilebilir. Bu ölçümler elle yapılmak zorunda değildir; kaynak kod ya da Java class dosyaları üzerinden coupling metriklerini ölçen araçlar vardır.
- Uyum örneği: veritabanı erişimi (ekrandan, 01:46:10 ve 01:58:45)
  - A→B→C sınıflarının nesneleri veritabanına yazılıp okunacaktır.
  - Tasarım I: veritabanı işlemlerinden sorumlu ayrı bir VT sınıfı; VT, A, B ve C nesnelerine sahiptir (çoklu olabileceği için elmaslı ilişki) ve veritabanına erişir.
  - Tasarım II: her sınıf kendisini veritabanına yazmaktan ve okumaktan sorumludur; A, B, C ile veritabanı arasında çift yönlü ilişki vardır.
  - Bağlaşım açısından Tasarım I'e üç ok eklendiği için II daha iyi görünür (fiziksel veritabanı bir sınıf olmadığından bağlaşıma sayılmadı).
  - Ama Tasarım II'de iş mantığı sınıflarına veritabanı sorumluluğu da yüklenir; uyum düşer. Veritabanı değiştiğinde (örneğin lisans ücreti nedeniyle SQL Server'dan MySQL'e geçiş ya da ilişkisel olmayan bir veritabanına geçiş) Tasarım II'de bütün sınıflar değişir, Tasarım I'de yalnız VT sınıfı değişir.
  - Doğru çözüm Tasarım I'dir. Bu tasarım iki katmanlı bir mimari kurar: iş mantığı katmanı (A, B, C) ve sistem (veritabanı) katmanı.
- Uyumun statik kod analiziyle ölçülmesi (ekrandan, 02:05:00 ve 02:07:55)
  - Bir sınıfın 3 üye alanı ve 4 metodu olsun. Her metottan kullandığı alana bir çizgi çizilir. En çok $4 \times 3 = 12$ çizgi olabilir; çizilen 6'dır. Uyum $= 6/12 = 0{,}5$.
  - Sınıf iki sınıfa bölünür (bir refactoring eylemi): birinci sınıfta 2 alan, 3 metot ve 5 çizgi vardır, uyum $= 5/6 \approx 0{,}83$. İkinci sınıfta 1 alan, 1 metot, 1 çizgi vardır, uyum $= 1/1 = 1{,}0$.
  - Ortalama uyum $= (0{,}83 + 1{,}0)/2 \approx 0{,}91$. Hoca sistemin ortalama uyumunun %90'ın biraz üstüne çıktığını söyledi.
  - Burada yalnız doğrudan kullanımlar sayıldı; dolaylı kullanımları da sayan çeşitlemeler vardır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.